PRACTICAL - 2

Distributed Chat Server using TCP Sockets in Java

Assignment Copy Write-up

Name	_____
Roll No.	_____
Class / Semester	_____
Date of Performance	_____
Date of Submission	_____

What to write in the assignment copy
Copy the sections in this order: Aim, Objectives, Requirements, Theory, Architecture, Working, Algorithms, Program, Execution, Sample Output, Error Handling, Result, Precautions, and Viva Questions. Draw the architecture diagram neatly with a pencil and ruler.

Department of Computer Science / Information Technology

Distributed Systems Laboratory

1. Aim

To design and implement a distributed multi-client chat application using Java TCP sockets, multithreading, and a Swing graphical user interface.

2. Objectives

- To establish reliable client-server communication using TCP.
- To allow multiple clients to connect to one central server simultaneously.
- To broadcast every valid message to all connected clients.
- To receive messages continuously without freezing the Swing user interface.
- To handle connection failures, client disconnections, invalid ports, and busy ports gracefully.

3. Software and Hardware Requirements

Category	Requirement
Operating System	Windows, macOS, or Linux
Software	Java Development Kit (JDK) 8 or later
Java Packages	java.net, java.io, java.util.concurrent, javax.swing, java.awt
Network	Loopback network for one computer, or LAN for different computers
Default Port	5050 (client and server must use the same port)

4. Theory

A **distributed system** is a collection of independent computers or processes that communicate through a network and appear to cooperate as one system. In this practical, a central server coordinates communication among several chat clients.

A **socket** is one endpoint of network communication. The server creates a **ServerSocket**, binds it to a port, and waits using **accept()**. Each client creates a **Socket** and connects using the server host name and the same port number.

TCP (Transmission Control Protocol) is connection-oriented and provides reliable, ordered delivery of bytes. Therefore, it is suitable for chat messages where correctness and ordering are important.

The server uses **multithreading**: every connected client is handled by a separate **ClientHandler** thread. A thread-safe set stores active clients. When one client sends a message, the server iterates over that set and broadcasts the message.

The client interface is built with **Java Swing**. Swing components must be updated on the Event Dispatch Thread. Hence, socket reading occurs on a background receiver thread, and received text is appended through **SwingUtilities.invokeLater()**. This keeps the UI responsive.

5. System Architecture

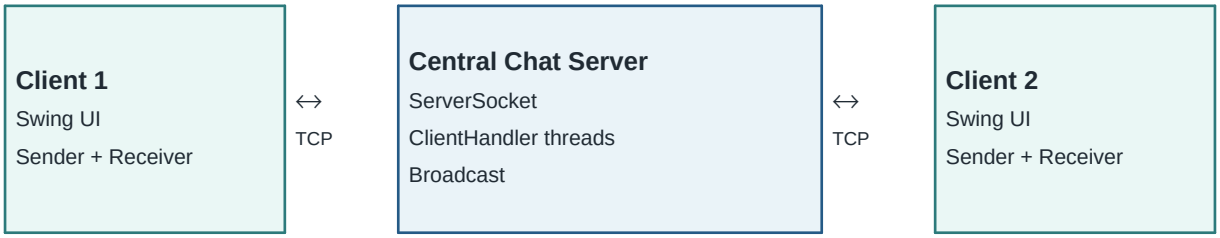

Figure 1: Centralized multi-client chat architecture. Each client maintains one TCP connection to the central server. The server broadcasts join, chat, and leave messages to all active clients.

6. Working of the System

1. The server validates the port, creates a ServerSocket, binds to port 5050, and waits for clients.
2. A client starts the Swing UI, asks for a username, and connects to the server in a background thread.
3. The first line sent by the client is the username. The server registers that client only after its streams are ready.
4. The server creates one ClientHandler thread for each connection and broadcasts the join message.
5. When the user clicks Send or presses Enter, the client transmits a non-empty message to the server.
6. The server prefixes the message with the username and sends it to every connected client.
7. Each client's receiver thread reads continuously and schedules UI updates on the Swing Event Dispatch Thread.
8. When a client disconnects, the server removes it from the active set, closes its socket, and broadcasts the leave message.

7. Algorithm

7.1 Server Algorithm

1. Read and validate the port number; use 5050 when no argument is supplied.
2. Create a ServerSocket, enable address reuse, and bind it to the selected port.
3. Repeat forever: accept a client socket and start a ClientHandler thread.
4. Inside the handler, create UTF-8 input and output streams and read the username.
5. Add the initialized handler to the thread-safe client set and announce that it joined.
6. Read messages continuously; ignore blank messages and broadcast valid messages.
7. On disconnection or error, remove the handler, announce departure, and close the socket.

7.2 Client Algorithm

1. Create the Swing window containing a non-editable chat area, input field, and Send button.
2. Ask for a username; close the window if the dialog is cancelled.
3. Connect to the server with a five-second timeout and send the username.
4. Enable input controls only after a successful connection.
5. Start a daemon receiver thread that continuously reads server messages.
6. Append each received message to the UI and automatically scroll to the newest line.
7. On Send or Enter, transmit a non-empty message and clear the input field. If disconnected, report the status once and disable the input controls.

8. Program

Program 1 - ChatServer.java

```
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class ChatServer {

    // Port 5000 is commonly used by AirPlay Receiver on macOS.
    private static final int DEFAULT_PORT = 5050;

    // Thread-safe set of connected clients
    private static final Set<ClientHandler> clients =
            ConcurrentHashMap.newKeySet();

    public static void main(String[] args) {
        int port = getPort(args);
        if (port == -1) {
            return;
        }

        try (ServerSocket serverSocket = new ServerSocket()) {
            serverSocket.setReuseAddress(true);
            serverSocket.bind(new InetSocketAddress(port));
            System.out.println("Chat Server started on port " + port);

            while (true) {
                Socket socket = serverSocket.accept();

                System.out.println(
                        "New client connected: " + socket.getInetAddress()
                );

                // One thread per connected client
                Thread clientThread = new Thread(new ClientHandler(socket), "chat-client-handler");
                clientThread.start();
            }

        } catch (BindException e) {
            System.err.println("Cannot start the chat server: port " + port + " is already in use.
");
            System.err.println("Try another port, for example: java ChatServer 5051");
            System.err.println("Then connect with: java ChatClient localhost 5051");
        } catch (IOException e) {
            System.err.println("Chat server stopped because of a network error: " + e.getMessage());
        }
    }

    private static int getPort(String[] args) {
        if (args.length == 0) {
            return DEFAULT_PORT;
        }

        try {
            int port = Integer.parseInt(args[0]);
            if (port < 1 || port > 65535) {
                throw new NumberFormatException();
            }
            return port;
        } catch (NumberFormatException e) {
            System.err.println("Port must be a number between 1 and 65535.");
            return -1;
        }
    }

    // Sends a message to all connected clients
    private static void broadcast(String message) {
        System.out.println(message);

        for (ClientHandler client : clients) {
            client.sendMessage(message);
        }
    }

    private static class ClientHandler implements Runnable {

        private final Socket socket;
        private BufferedReader reader;
        private PrintWriter writer;
```

Program 1 - ChatServer.java - continued (2/2)

```
        private String username;

        ClientHandler(Socket socket) {
            this.socket = socket;
        }

        @Override
        public void run() {
            try {
                reader = new BufferedReader(
                        new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8)
                );

                writer = new PrintWriter(
                        new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8),
                        true
                );

                // First line received from client is the username
                username = reader.readLine();

                // A client that disconnects before completing the handshake
                // must not appear as an "Anonymous" join/leave event.
                if (username == null) {
                    return;
                }

                username = username.trim();
                if (username.isEmpty()) {
                    username = "Anonymous";
                }

                // Register only after the connection has a working writer.
                // Otherwise an early broadcast can be lost for this client.
                clients.add(this);
                broadcast(username + " joined the chat.");

                String message;

                // Continuously receive messages from this client
                while ((message = reader.readLine()) != null) {
                    if (!message.trim().isEmpty()) {
                        broadcast(username + ": " + message);
                    }
                }

            } catch (IOException e) {
                System.out.println(
                        (username != null ? username : "A client")
                                + " disconnected."
                );
            } finally {
                clients.remove(this);

                if (username != null) {
                    broadcast(username + " left the chat.");
                }

                try {
                    socket.close();
                } catch (IOException ignored) {
                }
            }
        }

        void sendMessage(String message) {
            writer.println(message);

            if (writer.checkError()) {
                clients.remove(this);
                try {
                    socket.close();
                } catch (IOException ignored) {
                }
            }
        }
    }
}
```

Program 2 - ChatClient.java

```
import javax.swing.*;
import java.awt.*;
import java.io.*;
import java.net.*;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.atomic.AtomicBoolean;

public class ChatClient {

    private static final int DEFAULT_PORT = 5050;
    private static final int CONNECTION_TIMEOUT_MS = 5000;

    private JFrame frame;
    private JTextArea chatArea;
    private JTextField messageField;
    private JButton sendButton;

    private Socket socket;
    private BufferedReader reader;
    private PrintWriter writer;
    private volatile boolean connected;
    private final AtomicBoolean disconnectReported = new AtomicBoolean();

    public ChatClient(String serverAddress, int port) {
        createGUI();
        requestUsernameAndConnect(serverAddress, port);
    }

    private void requestUsernameAndConnect(String serverAddress, int port) {
        String username = JOptionPane.showInputDialog(frame, "Enter your name:");

        if (username == null) {
            frame.dispose();
            return;
        }

        if (username.trim().isEmpty()) {
            username = "Anonymous";
        }

        final String selectedUsername = username.trim();
        Thread connectionThread = new Thread(
                () -> connectToServer(serverAddress, port, selectedUsername),
                "chat-server-connector"
        );
        connectionThread.setDaemon(true);
        connectionThread.start();
    }

    private void connectToServer(String serverAddress, int port, String username) {
        try {
            Socket connectedSocket = new Socket();
            connectedSocket.connect(
                    new InetSocketAddress(serverAddress, port),
                    CONNECTION_TIMEOUT_MS
            );
            socket = connectedSocket;

            reader = new BufferedReader(
                    new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8)
            );

            writer = new PrintWriter(
                    new OutputStreamWriter(socket.getOutputStream(), StandardCharsets.UTF_8),
                    true
            );
```

Program 2 - ChatClient.java - continued (2/3)

```
            // Send username to server
            writer.println(username);
            if (writer.checkError()) {
                throw new IOException("Could not send username to server.");
            }

            connected = true;
            SwingUtilities.invokeLater(() -> {
                appendMessage("Connected to " + serverAddress + ":" + port + ".");
                messageField.setEnabled(true);
                sendButton.setEnabled(true);
                messageField.requestFocusInWindow();
            });

            // Separate thread continuously receives messages
            Thread receiverThread = new Thread(() -> {
                try {
                    String message;

                    while ((message = reader.readLine()) != null) {
                        final String receivedMessage = message;

                        SwingUtilities.invokeLater(() ->
                            appendMessage(receivedMessage)
                        );
                    }

                } catch (IOException e) {
                    // The cleanup below reports the disconnected state.
                } finally {
                    markDisconnected("Disconnected from server.");
                }
            });

            receiverThread.setDaemon(true);
            receiverThread.start();

        } catch (IOException e) {
            markDisconnected("Unable to connect to " + serverAddress + ":" + port + ".");
        }
    }

    private void createGUI() {
        frame = new JFrame("TCP Chat Application");

        chatArea = new JTextArea();
        chatArea.setEditable(false);
        chatArea.setLineWrap(true);
        chatArea.setWrapStyleWord(true);

        messageField = new JTextField();
        sendButton = new JButton("Send");
        messageField.setEnabled(false);
        sendButton.setEnabled(false);

        JScrollPane scrollPane = new JScrollPane(chatArea);

        JPanel bottomPanel = new JPanel(new BorderLayout(5, 5));
        bottomPanel.add(messageField, BorderLayout.CENTER);
        bottomPanel.add(sendButton, BorderLayout.EAST);

        frame.setLayout(new BorderLayout(5, 5));
        frame.add(scrollPane, BorderLayout.CENTER);
        frame.add(bottomPanel, BorderLayout.SOUTH);

        sendButton.addActionListener(e -> sendMessage());
        messageField.addActionListener(e -> sendMessage());
```

Program 2 - ChatClient.java - continued (3/3)

```
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setMinimumSize(new Dimension(400, 300));
        frame.setSize(500, 500);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
    }

    private void sendMessage() {
        String message = messageField.getText().trim();

        if (!message.isEmpty() && connected && writer != null) {
            writer.println(message);
            if (writer.checkError()) {
                markDisconnected("Disconnected from server.");
            } else {
                messageField.setText("");
                messageField.requestFocus();
            }
        }
    }

    private void appendMessage(String message) {
        chatArea.append(message + "\n");
        chatArea.setCaretPosition(chatArea.getDocument().getLength());
    }

    private void markDisconnected(String status) {
        if (!disconnectReported.compareAndSet(false, true)) {
            return;
        }

        connected = false;
        SwingUtilities.invokeLater(() -> {
            appendMessage(status);
            messageField.setEnabled(false);
            sendButton.setEnabled(false);
        });

        try {
            if (socket != null) {
                socket.close();
            }
        } catch (IOException ignored) {
        }
    }

    public static void main(String[] args) {
        String serverAddress = args.length > 0 ? args[0] : "localhost";
        int port = DEFAULT_PORT;

        if (args.length > 1) {
            try {
                port = Integer.parseInt(args[1]);
                if (port < 1 || port > 65535) {
                    throw new NumberFormatException();
                }
            } catch (NumberFormatException e) {
                System.err.println("Port must be a number between 1 and 65535.");
                return;
            }
        }

        final int selectedPort = port;
        SwingUtilities.invokeLater(() -> new ChatClient(serverAddress, selectedPort));
    }
}
```

9. Compilation and Execution

Open a terminal in the **src** folder and compile both programs:

```
javac ChatServer.java ChatClient.java
```

Start the server in the first terminal:

```
java ChatServer
```

Start one or more clients in separate terminals:

```
java ChatClient
```

For a server on another computer, pass its IP address and port:

```
java ChatClient 192.168.1.10 5050
```

If port 5050 is busy, select another port for **both** programs:

```
java ChatServer 5051
java ChatClient localhost 5051
```

10. Sample Output / Observation

10.1 Server Terminal

```
Chat Server started on port 5050
New client connected: /127.0.0.1
Alice joined the chat.
New client connected: /127.0.0.1
Bob joined the chat.
Alice: Hello Bob!
Bob: Hello Alice!
Alice left the chat.
```

10.2 Client 1 Window

```
Connected to localhost:5050.
Alice joined the chat.
Bob joined the chat.
Alice: Hello Bob!
Bob: Hello Alice!
```

10.3 Client 2 Window

```
Connected to localhost:5050.
Bob joined the chat.
Alice: Hello Bob!
Bob: Hello Alice!
Alice left the chat.
```

11. Important Error and Its Solution

Error: java.net.BindException: Address already in use
This error means another process is already listening on the selected TCP port. On macOS, Control Center / AirPlay Receiver commonly uses port 5000. Therefore, this project uses port 5050 by default. If 5050 is also occupied, start the server on another free port, such as 5051, and pass exactly the same port to every client.

The corrected server catches **BindException** separately. Instead of printing a long stack trace, it displays commands that show how to start the server and clients on another port.

12. Test Cases

No.	Test	Expected Result
1	Start server on free port	Server starts and waits for connections
2	Connect two clients	Both receive join notifications
3	Send from Client 1	Message appears in both client windows
4	Send a UTF-8 message	Text is delivered without corruption
5	Close one client	Remaining clients receive the leave message
6	Start a second server on the same port	Clear busy-port instructions are displayed
7	Enter an invalid port	Program rejects values outside 1 to 65535

13. Result

Result
The distributed multi-client chat application was implemented successfully using Java TCP sockets, multithreading, and Swing. Multiple clients can connect to a central server and exchange messages in real time. Continuous receiving, message broadcasting, UTF-8 text, join/leave notifications, invalid input, disconnections, and busy-port errors are handled correctly.

14. Precautions

- Start the server before starting any client.
- Use the same port number on the server and every client.
- Use localhost when both programs run on the same computer; otherwise use the server's LAN IP address.
- Do not run two server instances on the same IP address and port.
- Keep blocking network operations outside the Swing Event Dispatch Thread.
- Update Swing components using SwingUtilities.invokeLater().
- Close sockets and remove disconnected clients to avoid resource leaks.
- Allow the Java application through the firewall when testing across different computers.

15. Viva Questions and Answers

These short answers are suitable for practical viva preparation.

1. What is a socket?

Answer: A socket is one endpoint of a two-way network communication link between two programs.

2. Why is TCP used for this chat application?

Answer: TCP provides connection-oriented, reliable, and ordered delivery of data.

3. What is the difference between ServerSocket and Socket?

Answer: ServerSocket listens for incoming connections. Socket represents an established connection used to send and receive data.

4. Why is multithreading required on the server?

Answer: It allows the server to communicate with several clients concurrently instead of blocking on one client.

5. What does accept() do?

Answer: It blocks until a client connects, then returns a Socket for that client connection.

6. What is broadcasting?

Answer: Broadcasting means sending one message to every currently connected client.

7. Why is ConcurrentHashMap.newKeySet() used?

Answer: It provides a thread-safe set because client threads can add, remove, and iterate over active clients concurrently.

8. Why is PrintWriter created with autoFlush=true?

Answer: It flushes each println() immediately so chat messages are transmitted without waiting for the buffer to fill.

9. Why is readLine() used?

Answer: The application treats each newline-terminated string as one complete username or chat message.

10. What is the Swing Event Dispatch Thread?

Answer: It is the thread responsible for processing Swing events and updating Swing components safely.

11. Why does the receiver run in a background thread?

Answer: readLine() is blocking. Running it on the UI thread would freeze the chat window while waiting for a message.

12. What does SwingUtilities.invokeLater() do?

Answer: It schedules a UI update on the Event Dispatch Thread.

13. What causes BindException: Address already in use?

Answer: Another process has already bound a listening socket to the same IP address and port.

14. How is the BindException solved in this project?

Answer: The default port is 5050, and a custom free port can be passed to both the server and clients.

15. Why must the client and server use the same port?

Answer: The port identifies the server application. A client connecting to another port reaches a different service or no service.

16. What is the purpose of a connection timeout?

Answer: It prevents the client from waiting indefinitely when the server address is unreachable.

17. Why is UTF-8 used?

Answer: UTF-8 allows messages containing characters from many languages to be transmitted consistently.

18. Is this system peer-to-peer?

Answer: No. It follows a centralized client-server architecture because every message passes through the central server.

19. What happens when a client disconnects?

Answer: Its handler is removed from the active set, the socket is closed, and a leave message is broadcast.

20. How can clients on different computers connect?

Answer: Run the server on one computer, use its LAN IP address in each client, keep the same port, and permit the connection through the firewall.

Teacher's Remarks:

_____

Signature: _____